

Alexandra Chase

amchas26@g.holycross.edu

CSCI 324 Programming Languages

27 April 2026

Prolog Lab

0. I have read and understand the entirety of Chapter 16 in our textbook.

1. A Horn Clause is a form of causal propositional statements. It can take the following forms:

- (Headed) a collection of conjoined facts implying another singular fact. This indicates a relationship between a specific set of conditions and one singular fact.

`Father(Charlie, Bob)  $\wedge$  Cousin(Alice, Bob)  $\rightarrow$  Uncle(Charlie, Alice)`

- (Headed) a single fact implying another singular fact to indicate relationship. This indicates a relationship between a specific condition and one singular fact.

`Father(Charlie, Bob)  $\rightarrow$  Son(Bob, Charlie)`

- (Headless) declaration of an unconditional assertion or assumed fact.

`Father(Charlie, Bob)`

Horn Clauses are essential for prolog's functionality as it allows the program to run in linear time.

2. Having “no intrinsic semantics” means that the functions and relational terms used in the propositions do not carry any inherent meaning to the Prolog system. The propositions mean whatever the programmer decides they mean. For example, `Caught(Bob, Fish)` is meaningless to the program parser. The programmer has to decide how the `Caught()` function operates and what the relationship between the parameters actually means.

3. These statements are not considered assignment statements like those in languages like Java. In Java, an assignment statement using `=` bounds a value to memory, while modeling it as a variable. You then have the opportunity to retrieve said variable, modify it, and put it back to memory. The `is` operator is quite different: it does not assign a value to a memory location. Instead, it is a relational statement attempting to satisfy a logical goal. It takes the expression on the right hand side and binds the result to the left hand variable. That being said, you cannot modify or overwrite the variable later on.

- The statement `A is B / 17 + C` is a legal statement, but will work only if the variables B and C are already assigned with values, and A is not assigned. If A is already assigned, or if B and C are not, the statement fails to be satisfied.
- The statement `Sum is Sum + 17` highlights a scenario where the `is` operator is confused with an assignment statement. Because the `is` operator requires the right side

variables to be assigned and the left side variable to be unassigned, a variable cannot be evaluated and assigned in the exact same proposition. Thus this statement will never be satisfied.

4. In Prolog, a list is represented as a sequence of atoms enclosed in square brackets and separated by commas. For example, a simple list of atoms can be represented as `[apple, prune, grape, kumquat]`. Lists can also be represented using head and tail notation separated by a vertical bar, such as `[New_Head | New_Tail]` or `[apricot, peach, pear | []]`

5.

```
> question.pro
% Alexandra chase
% amchas26@g.holycross.edu
% CSCI 324
% 27 April 2026
% Question Prolog Lab

what([], X, X). % BASE CASE if the first list is empty, then
what([X | R], Y, Z) :-
    member(X, Y), % checks if the head of the first list is a
    !, % if so, then cut it from the first list
    what(R, Y, Z). % and recurse through the rest of the first
what([X | R], Y, [X | Z]) :- what(R, Y, Z). % when X is not a member of Y, it becomes t
```

This function takes three lists, X (list1), Y (list2), and Z (ANS). First, it checks the base case: is the list1 empty? If so, then the contents of list2 are prepended to the ANS list. If the base case is not met, it checks the first condition: is the head of X a member of list Y? If so, it cuts the atom from list X and recursively moves on to the next atom in the tail. If not, then we move onto the second condition: if the head of X is NOT a member of the list Y, then it is prepended to the answer list Z. The product of this program is an answer list, in which all the atoms of list Y are at the end and all of the unique atoms in X in the front. This is because the items are prepended as Prolog traces back up the steps from the base case.

```
| ?- what([2,4],[1,2,3,4,5], ANS).
```

```
ANS = [1,2,3,4,5]
```

```
(1 ms) yes
```

```
| ?-
```

trace: In this instance, all the values of X are contained in Y. This means in both instances we use the first condition. We cut 2 the first iteration, we cut 4 from X the second iteration, and reach the base case in the third iteration. At that point, we add everything in Y to the answer list. At that point, we trace back up the steps that led us here. There are no unique items in list X to prepend to ANS, so we end up just with the original list of Y.

```
| ?- what([1,2,3,4,5], [2,4], ANS).
```

```
ANS = [1,3,5,2,4]
```

```
yes
```

```
| ?- halt.
```

trace: In this example, many of the values in X are unique, though not all. For the iterations examining 1,3, and 5 as the head of X, we use the second condition. This prepends 1,3, and 5 to the beginning of the ANS list. However, prolog recurses all the way down to the base case before actually completing the prepend. That is why these numbers are first, since they are prepended as prolog traces back up the steps. The 2 and 4 are cut from X since they are members of Y and fulfill the first condition. The base case is reached once X is empty, which is where 2,4 gets added to the ANS list.